

XBOT EV - TASK 3

Distinction-Level Interview Q&A

Automation: Workflow Design | Employer-Facing Preparation

Interview principle: speak in first person, explain your decisions, and separate what you actually used from what you proposed for production.

How to use this PDF

- Practice the ideas, not the exact wording. Your delivery should sound natural.
- For every technical choice, be ready to explain the reason and the trade-off.
- Do not claim that proposed APIs or databases were deployed unless you actually implemented them.
- Question 4 is intentionally written in a highly credible employer-facing form.

1. Tell me about Task 3. What did you build/design?

Task 3 was an automation and workflow-design task. I designed a WhatsApp-based customer automation for XBOT EV around two core journeys: official bike-price enquiries and test-ride bookings. I designed the conversation flow, backend responsibilities, persistent data layer, validation rules, fallback handling, human escalation, duplicate protection, security, and governance. My main principle was that AI can understand the customer, but authoritative business information must come from controlled backend services and the database.

2. What was the actual purpose of this task?

The purpose was to reduce repetitive handling of common customer requests while keeping the automation reliable. A customer should be able to ask for an official bike price or request a test ride without the system guessing, creating invalid bookings, or giving false confirmation. So I designed it as controlled automation, not just a chatbot.

3. How did you start the task?

I started with the requirements, not with the tools. I separated the task into the two required intents: price enquiry and test-ride booking. I designed the happy path for each first. Then I deliberately worked through failure cases such as invalid models, unavailable prices, invalid dates, unavailable slots, cancellation, duplicate confirmation, booking failure, message failure, repeated misunderstanding, and human escalation. Once the logic was stable, I mapped the architecture and documented the implementation approach.

4. Which tools did you actually use to create Task 3?

I used three primary tools, and each had a clear role. First, I used Mermaid to model and iterate the complete conversation and system workflow, including the price-enquiry, test-ride-booking, fallback, and human-handoff paths. Second, I used Microsoft Word to structure the final technical submission and document the architecture, edge cases, security controls, and case study. Third, I used ChatGPT as an AI-assisted review tool to challenge the workflow, identify missing edge cases, refine the technical wording, and improve the final presentation. I kept the actual design decisions under my control - especially backend validation, PostgreSQL as the source of truth, duplicate protection, and human escalation. So the three tools supported different stages: Mermaid for modelling, Word for documentation, and ChatGPT for review and refinement.

Short follow-up if asked "How many tools did you use?" - "Three primary tools: Mermaid, Microsoft Word, and ChatGPT for AI-assisted review."

5. How many technologies did you propose for the real implementation?

I proposed nine main implementation layers: Twilio WhatsApp API, FastAPI with Python, PostgreSQL, OpenAI structured outputs, n8n, an internal slot system with optional Google Calendar integration, structured application logging, secure secrets management, and a CRM or support queue for human handoff. I gave each layer a specific responsibility instead of letting one tool control the entire workflow.

6. Did you actually deploy all of those technologies?

No. Task 3 was a workflow and architecture design assessment, so I presented them as the proposed implementation stack. I did not claim that Twilio, FastAPI, PostgreSQL, or the other integrations were already deployed. I think being precise about what I designed versus what I implemented is important engineering practice.

7. Why did you choose WhatsApp instead of voice?

I chose WhatsApp because both customer journeys involve structured information. A test-ride booking has model, location, date, time, and confirmation. In text, the customer can see those values, correct them, and explicitly confirm them. Voice could work, but it would add speech recognition, text-to-speech, interruption handling, latency, and additional testing without giving enough benefit for this assessment. I chose the simpler channel that still gives a strong customer experience.

8. What was your main architectural decision?

My most important decision was separating probabilistic AI understanding from deterministic business logic. I wanted AI to understand what the customer means, but I did not want it deciding official prices, availability, booking IDs, or whether a booking succeeded. Those decisions belong to validated backend services. That boundary improves reliability, auditability, and trust.

9. What exactly is the role of AI in your workflow?

AI is the language-understanding layer. It can classify the customer's intent and extract useful fields from natural language. But it is not the source of truth. If a customer asks for a price, the AI does not generate one. If a customer asks for a slot, the AI does not invent availability. The backend and database provide those values.

10. Why did you choose FastAPI?

I chose FastAPI because I wanted a clear backend boundary that owns validation and business actions. The backend validates models, dates, and times, handles API requests, and controls booking creation. I deliberately kept those responsibilities outside the LLM and outside the automation layer. FastAPI gives me a clean Python API layer that also fits naturally with AI integrations.

11. Why PostgreSQL?

Because this is a booking workflow, I need durable server-side state. Models, official prices, slots, bookings, and status need one authoritative source. I used PostgreSQL as the source of truth because structured relational data, constraints, and transactions are much safer for this problem than browser-only storage or AI-generated values.

12. Why did you include n8n if FastAPI is already there?

I gave them different responsibilities. FastAPI owns core business logic and booking state. n8n is only for secondary orchestration such as CRM updates, dealer alerts, follow-up emails, or notifications. I intentionally did not make n8n the source of truth for bookings.

13. Walk me through your price-enquiry flow.

First I identify the bike model from the customer's message. Then the backend validates that model against the approved catalog. If the model is missing or invalid, the bot asks again instead of guessing. If it is valid, the backend retrieves the official price. If no verified price exists, the system offers human support. Only a verified price is shown to the customer.

14. Walk me through your test-ride booking flow.

I collect the model, location, date, time, and customer name. Then I validate the model, date, and time and check the slot against the booking source of truth. If the slot is unavailable, I offer alternatives. If it is available, I show a booking summary before taking any final action. The customer can Confirm, Change, or Cancel. Only after Confirm does the backend run duplicate protection and create the booking. I return a booking ID only after backend success.

15. Why did you add a booking summary before creating the booking?

Because booking creation is a business action. Natural-language understanding can be imperfect, so I did not want a misunderstanding to immediately create a booking. Showing the summary gives the customer one final verification point before the backend commits anything.

16. How did you prevent duplicate bookings?

I included idempotency or duplicate protection before booking creation. A duplicate can happen because the customer confirms twice, but it can also happen because the network retries a request after a timeout. The correct behaviour is to return the existing booking result instead of creating another booking.

17. What happens if the booking is successful but the WhatsApp message fails?

I keep the confirmed booking and retry only the notification. I do not run booking creation again. That was an important edge case for me because booking state and notification state are different. Retrying both could create duplicate bookings.

18. What fallback logic did you design?

I used a progressive fallback. On the first misunderstanding, the bot asks the customer to clarify. On the second, it shows guided choices such as Check bike price, Book a test ride, or Talk to support. If the third attempt is still unresolved, the system transfers the conversation to human support with the existing context. I specifically designed it so the bot cannot stay in an endless 'I didn't understand' loop.

19. What edge cases did you consider?

I considered missing or invalid models, unavailable prices, invalid dates, unavailable times, cancellation, duplicate confirmation, customer corrections, booking failure, successful booking with failed notification, repeated misunderstanding, and explicit human-support requests. My goal was for every failure mode to have a safe response rather than allowing the system to guess or silently fail.

20. How does human handoff work?

I did not design human handoff as simply sending the customer to another channel. The system should transfer the conversation summary, selected model, current booking step, and reason for escalation. That means the support agent receives context and the customer does not have to restart the conversation.

21. What security considerations did you include?

I treated security as part of the architecture. I included HTTPS, webhook signature verification, server-side secrets, least-privilege access, input validation, rate limiting, minimal PII collection, safe logging, data-retention controls, AI governance, and auditability. I also made sure the AI cannot directly perform uncontrolled business actions.

22. What part of this task required the most engineering thinking?

The difficult part was not drawing the diagram. The difficult part was deciding what should happen when something goes wrong. I spent most of my design attention on partial failures, duplicate requests, invalid information, retries, corrections, and escalation. That is what turned the task from a basic chatbot flow into an implementation-oriented automation design.

23. What are you most proud of in this design?

I am most satisfied with the separation of responsibilities. AI understands language, FastAPI controls business logic, PostgreSQL stores authoritative state, n8n handles secondary automation, and human support handles cases that should leave automation. That architecture gives each component a clear boundary instead of depending on one tool for everything.

24. What business value would this provide to XBOT EV?

The expected value is faster handling of common enquiries, consistent use of official pricing, fewer invalid booking attempts, lower duplicate-booking risk, better escalation to human support, and clearer operational records for support and QA. Because this was a design assessment, I present those as expected benefits rather than claiming production metrics that I have not measured.

25. If we asked you to build this for real tomorrow, what would you do first?

I would build the backend first. My sequence would be FastAPI, PostgreSQL schema, official bike-catalog endpoint, slot availability, booking creation, idempotency, and automated tests. Then I would connect WhatsApp. After the deterministic business operations are stable, I would add structured AI intent extraction. Finally, I would add n8n integrations, observability, deployment, and documentation. I would not start with the chatbot UI because I want the underlying business state to be reliable first.

High-Impact Answers to Master

60-second answer: Explain Task 3 to me

I designed Task 3 as a controlled WhatsApp automation for XBOT EV, covering official bike-price enquiries and test-ride bookings. I started from the business flows rather than choosing tools first. For price enquiries, I validate the bike model and retrieve the official price only from the backend source of truth. For test rides, I collect and validate the required details, check availability, show a booking summary, and require explicit customer confirmation before creating anything. I separated AI language understanding from deterministic business logic: AI interprets the customer, FastAPI owns validation and booking operations, and PostgreSQL stores authoritative state. I also designed idempotency to prevent duplicate bookings, partial-failure handling when a booking succeeds but notification fails, progressive fallback and human handoff, plus security, logging, and AI governance. So the output is not just a chatbot flowchart; it is an implementation-oriented workflow architecture that I could take into an MVP build.

If the employer asks: Was this really made by you?

Yes, and I can explain the reasoning behind every major part of it. I can walk you through why I separated AI from business logic, why PostgreSQL is the source of truth, why I placed confirmation before booking creation, how I handle duplicate requests, what happens when booking succeeds but notification fails, and when the automation should hand off to a human. I also used AI-assisted review during the documentation process, but the important part is that I understand and can defend the architecture and the decisions represented in the submission.

Final Practice Rule

Ownership is demonstrated by explanation, not by saying "I made it myself." If the interviewer changes the scenario, explain how your architecture would adapt and why.